

Crowd Management System Diagrams

This document contains the core architectural and operational diagrams for the CMS project, generated based on the system's codebase (`app.py` , `yolo_inference.py` , `room_capacity.py`).

1. Class Diagram

The class diagram outlines the main components of the backend. It models the `FastAPI` application endpoints, the `YOLOInference` processing engine, and the MiDaS-based `RoomCapacityController` .

```
classDiagram
class FastAPIServer {
    +app : FastAPI
    +index(request)
    +live_camera(request)
    +start_camera(request)
    +stop_camera()
    +camera_feed()
    +get_camera_stats()
    +upload(video)
    +upload_image(image, area, scale, ...)
    +batch_process(files, ...)
}

class YOLOInference {
    -device : str
    -model : YOLO
    -tracker : DeepSort
    -density_map : ndarray
    -latest_frame : ndarray
    +__init__(model_path)
    +set_heatmap_enabled(state : bool)
    +set_zoom_cell(row : int, col : int)
    +get_zoomed_subimage() : ndarray
    +draw_overlays(frame, people_count, density) : ndarray
    +process_video(input_path, output_path)
    +process_image(image_path, output_path)
    +get_density_level(density) : str
    +get_alert_status(density, people_count) : str
    +estimate_area_from_detections(width, height, detections) : float
}

class RoomCapacityController {
    -midas : torch.nn.Module
    -transform : torchvision.transforms
    +estimate_room_capacity(image_path : str) : dict
}

FastAPIServer --> YOLOInference : "Instantiates and queries"
FastAPIServer --> RoomCapacityController : "Calls for depth analysis"
```

2. Sequence Diagram

This sequence diagram depicts the flow of a client uploading an image for crowd density and safety analysis over a specific area.

```
sequenceDiagram
    actor User
    participant Browser
    participant FastAPIServer
    participant YOLOInference

    User->>Browser: Uploads Image + Selects Area
    Browser->>FastAPIServer: POST /upload_image (image, area_sqm, bounds)
    FastAPIServer->>FastAPIServer: Save image to temporary static directory
    FastAPIServer->>FastAPIServer: process_image_with_area_analysis()
    FastAPIServer->>YOLOInference: model(frame)
    YOLOInference-->>FastAPIServer: Object Detections (bounding boxes)
    FastAPIServer->>FastAPIServer: Count "Person" classes inside selected boundaries
    alt Area not provided manually
        FastAPIServer->>YOLOInference: estimate_area_from_detections()
        YOLOInference-->>FastAPIServer: Estimated Area (sqm)
    end
    FastAPIServer->>FastAPIServer: Compute Density & Set Safety Status
    FastAPIServer->>FastAPIServer: Draw Overlays, Bounding Boxes, & Statistics onto Frame
    FastAPIServer->>FastAPIServer: Save processed output
    FastAPIServer-->>Browser: JSON Response (Stats + Processed Image Path)
    Browser-->>User: Render updated UI with Analytics
```

3. Activity Diagram

The activity diagram models the core repetitive flow of the **live camera feed processing thread**, handling real-time frame evaluation and stream broadcasting.

```
flowchart TD
    Start([Start]) --> InitCamera[InitCamera[Initialize Camera & Load YOLO weights]]
    InitCamera --> CaptFrame[CaptFrame[Capture Frame from Camera Stream]]
    CaptFrame --> YoloInference[YoloInference[Run YOLO Object Detection]]
    YoloInference --> ForEachDet{ForEachDet{For Each Detection}}

    ForEachDet -->|Person Detected| IncCount[IncCount[Increment People Count]]
    IncCount --> UpdateDensity[UpdateDensity[Update Internal Density Map]]
    UpdateDensity --> ForEachDet

    ForEachDet -->|Other Class Detected| Skip[Skip[Draw Bounding Box & Ignore Count]]
    Skip --> ForEachDet

    ForEachDet -->|All Detections Processed| CalcArea{CalcArea{Is Area Provided?}}

    CalcArea -->|Yes| UseArea[UseArea[Use Provided Square Meter Area]]
    CalcArea -->|No| EstArea[EstArea[Estimate Camera FOV Area]]
```

```

UseArea --> CalcDensity[Calculate Crowd Density]
EstArea --> CalcDensity

CalcDensity --> DetermineStatus[Evaluate Density Level against Status Thresholds]
DetermineStatus --> DrawOverlay[Render Stats, Status, and Custom HUD on Frame]
DrawOverlay --> Send[Encode Frame to MJPEG buffer]
Send --> Broadcast[Yield to /camera_feed connected clients]

Broadcast --> ActiveCheck{Is Camera Still Active?}
ActiveCheck -->|Yes| CaptFrame
ActiveCheck -->|No| ReleaseResources[Release Camera & Threads]
ReleaseResources --> End([End])

```

4. State Chart Diagram

The state chart diagram illustrates the state transitions for the primary processing pipelines: camera stream, batched video processing, and static image analysis.

```

stateDiagram-v2
    [*] --> Idle

    Idle --> CameraProcessing : POST /start_camera
    Idle --> VideoProcessing : POST /upload
    Idle --> ImageAnalysis : POST /upload_image

    state CameraProcessing {
        [*] --> InitializingCamera
        InitializingCamera --> LoadingYOLOModel
        LoadingYOLOModel --> CapturingFrames
        CapturingFrames --> DetectingObjects
        DetectingObjects --> AnalyzingDensity
        AnalyzingDensity --> StreamingMJPEG
        StreamingMJPEG --> CapturingFrames
        StreamingMJPEG --> Stopped : POST /stop_camera
    }

    CameraProcessing --> Idle : Camera Stopped

    state VideoProcessing {
        [*] --> DecodingVideo
        DecodingVideo --> FrameByFrameExtraction
        FrameByFrameExtraction --> TrackingObjects
        TrackingObjects --> DensityCalculation
        DensityCalculation --> EncodingVideo
        EncodingVideo --> SavedToProcessed
    }

    VideoProcessing --> Idle : Processing Complete

    state ImageAnalysis {

```

```

    [*] --> LoadingImage
    LoadingImage --> YOLORecognition
    YOLORecognition --> CapacityEstimation
    CapacityEstimation --> OverlayGeneration
    OverlayGeneration --> SavedToProcessed
}

ImageAnalysis --> Idle : Analysis Complete

```

5. Collaboration (Communication) Diagram

This diagram maps out the interactive message flow passed between objects/components during asynchronous video and camera analysis using a UML-style flowchart.

```

flowchart TD
    1(1: Process Request) --> |User / Client| 2[FastAPI Router]
    2 --> |2: Handle Request Async| 3[Background Thread / process_video]
    3 --> |3: cv2.VideoCapture| 4[Input Media Stream]
    3 --> |4: model.predict| 5[YOLOv11 Inference Model]
    5 --> |5: Return YOLO Detections| 3
    3 --> |6: tracker.update| 6[DeepSort Object Tracker]
    6 --> |7: Return Tracked Paths| 3
    3 --> |8: draw_overlays| 3
    3 --> |9: stream frame| 7[Output / MJPEG Buffer]
    7 --> |10: Serve Media| 1

```